C++ Mini Project

Student Name: Harsh Kumar

Roll No: 150096725105

B.Tech CSE 25-29 **[Sam Altman]**

Semester I I

Sprint I

Subject: C++ Programming

Group 9 (Harsh Kumar, Kunwar Bhosle, Kaushal Rajmandai, Saksham)

🗂️ TABLE OF CONTENTS

1. Problem Statement
2. Background / Scenario
3. Objective / Learning Goals
4. Tools Used
5. Implementation (Shell Script)
6. Output
7. References

🔰 **PROBLEM STATEMENT**

→ **Pattern Generator Tool**

Generate different patterns on User inputs.

Using Nested Loops and Logic.

Introduction

Pattern printing is one of the most fundamental exercises in learning any programming language. It helps beginners develop a clear understanding of how loops work, how output is controlled character by character, and how nested structures interact with each other.

This project, the Pattern Generator Program, is an interactive console-based application developed in C++. It allows users to select from three categories of patterns — Star Patterns, Number Patterns, and Continuous Number Patterns — and then further choose the shape of the pattern from four options: Square, Triangle, Left Triangle, and Right Triangle.

The program uses a simple text-based menu system to guide the user through their selections and then displays the chosen pattern directly in the console window. It is designed to be a clean, beginner-level demonstration of core C++ programming skills.

---

2. Aim & Objectives

2.1 Aim

The aim of this project is to design and implement a menu-driven Pattern Generator Program in C++ that can display a variety of visual patterns — made of stars or numbers — in different geometric shapes, demonstrating the use of nested loops and conditional logic.

2.2 Objectives

The specific objectives of this project are:

- To understand and apply nested for-loops to generate row-and-column based patterns.
- To implement a multi-level, menu-driven console interface using cin and cout.
- To generate 12 distinct pattern variations across 3 types and 4 shape options.
- To demonstrate the use of conditional statements (if) for routing program execution.
- To use a sequential counter variable for continuous number patterns.
- To produce readable, structured, and well-commented C++ source code.

- To reinforce understanding of the relationship between loop variables and output positioning.

Theory & Concepts Used

Nested Loops

A nested loop is a loop inside another loop. In this program, the outer loop iterates over rows and the inner loop iterates over columns. For every iteration of the outer loop, the inner loop runs completely, printing characters across one row. This combination creates the row-column grid structure needed for patterns.

Conditional Statements

The if statement is used extensively to direct program flow based on the user's menu choices. The variable t holds the pattern type (1, 2, or 3), and c holds the shape choice (1 to 4). Nested if blocks ensure the correct pattern-drawing code is executed for each combination.

Console Input / Output

The program uses cout (console output) to print characters and newlines, and cin (console input) to read the user's integer selections from the keyboard. The endl manipulator is used to move to the next line after each row of the pattern.

Sequential Counter Variable

For Continuous Number Patterns, an integer variable num is initialized to 1 before the outer loop. It is incremented (num++) every time a character is printed. Because num is declared outside both loops, its value persists across rows, producing a continuous counting sequence throughout the entire pattern.

Space Alignment

For right-aligned triangle shapes, a third inner loop prints blank spaces before the actual pattern characters. The number of spaces decreases as the row number increases (5 - i spaces in row i), creating the right-aligned staircase visual effect.

Algorithm

5.1 Main Program Algorithm

1. START
2. Display main menu: (1) Star Pattern (2) Number Pattern (3) Continuous Number Pattern
3. Read user choice into variable t.
4. Display shape sub-menu: (1) Square (2) Triangle (3) Left Triangle (4) Right Triangle
5. Read shape choice into variable c.
6. Based on values of t and c, execute the appropriate nested loop block.
7. Print the pattern to the console.
8. END

5.2 Algorithm for Left Triangle (Star) — Example

1. Set outer loop: i from 1 to 5.
2. Set inner loop: j from 1 to i.
3. Print '* ' in each inner loop iteration.
4. After inner loop ends, print newline (endl).
5. Repeat until outer loop completes.

5.3 Algorithm for Continuous Number Pattern (Left Triangle)

1. Initialize num = 1.
2. Set outer loop: i from 1 to 5.
3. Set inner loop: j from 1 to i.
4. Print num followed by a space; increment num.
5. After inner loop ends, print newline.
6. Repeat until outer loop completes.

Source Code

```
#include <iostream>
using namespace std;
```

```
int main()
{
    int i,j,t,c;
    cout<<"-----------------------------------------------"<
<endl;
    cout<<"Enter the type of Pattern you want"<<endl;
    cout<<"-----------------------------------------------"<
<endl;
    cout<<" 1. Star Pattern \n 2. Number Pattern \n 3. Cont
inuous Number Pattern"<<endl;
    cout<<"Enter your choice: ";
    cin>>t;
    if(t==1)
    {
        cout<<"-------------------------------------------
--"<<endl;
        cout<<"Enter the shape of Pattern you want"<<endl;
        cout<<"-------------------------------------------
--"<<endl;
        cout<<" 1. Square \n 2. Triangle \n 3. Left Triangl
e \n 4. Right Triangle"<<endl;
        cout<<"Enter your choice: ";
        cin>>c;
        cout<<"-------------------------------------------
--"<<endl;
        if(c==1)
        {
            for(i=1;i<=5;i++)
            {
                for(j=1;j<=5;j++)
                {
                    cout<<"* ";
                }
                cout<<endl;
            }
        }
        if(c==2)
```

```
        {
            for(i=5;i>=1;i--)
            {
                for(j=1;j<=5;j++)
                {
                    if(j>=i)
                    cout<<"* ";
                    else
                    cout<<" ";
                }
                cout<<endl;
            }
        }
        if(c==3)
        {
            for (i=1;i<=5;i++)
            {
                for (j=1; j<=i;j++)
                {
                    cout << "* ";
                }
                cout<<endl;
            }
        }
        if(c==4)
        {
            for (i=1;i<=5;i++)
            {
                for (int k=5-i;k>0;k--)
                    cout << " ";
                for(j=1;j<=i;j++)
                {
                    cout<<"*";
                }
                cout<<endl;
            }
        }
    }
```

```
    if(t==2)
    {
        cout<<"-----------------------------------------
--"<<endl;
        cout<<"Enter the shape of Pattern you want"<<endl;
        cout<<"-----------------------------------------
--"<<endl;
        cout<<" 1. Square \n 2. Triangle \n 3. Left Triangl
e \n 4. Right Triangle "<<endl;
        cout<<"Enter your choice: ";
        cin>>c;
        cout<<"-----------------------------------------
--"<<endl;
        if(c==1)
        {
            for(i=1;i<=5;i++)
            {
                for(j=1;j<=5;j++)
                {
                    cout<<i<<" ";
                }
                cout<<endl;
            }
        }
        if(c==2)
        {
            for (i=1;i<=5;i++)
            {
                for (j=1;j<=5-i;j++)
                {
                cout << " ";
                }
                for (j=1;j<=i;j++)
                {
                cout << i << " ";
                }
            cout << endl;
            }
```

```
        }
        if(c==3)
        {
            for (i=1;i<=5;i++)
            {
                for (j=1; j<=i;j++)
                {
                    cout << i;
                }
                cout<<endl;
            }
        }
        if(c==4)
        {
            for (i=1;i<=5;i++)
            {
                for (int k=5-i;k>0;k--)
                    cout << " ";
                for(j=1;j<=i;j++)
                {
                    cout<<i;
                }
                cout<<endl;
            }
        }
    }
    if(t==3)
    {
        cout<<"-----------------------------------------
--"<<endl;
        cout<<"Enter the shape of Pattern you want"<<endl;
        cout<<"-----------------------------------------
--"<<endl;
        cout<<" 1. Square \n 2. Triangle \n 3. Left Triangl
e \n 4. Right Triangle"<<endl;
        cout<<"Enter your choice: ";
        cin>>c;
```

```
        cout<<"-----------------------------------------
--"<<endl;
        if(c==1)
        {
            int num = 1;
            for(i=1;i<=3;i++)
            {
                for(j=1;j<=3;j++)
                {
                    cout<<num<<" ";
                    num++;
                }
                cout<<endl;
            }
        }
        if(c==2)
        {
            int num=1;
            for (i=1;i<=3;++i)
            {
                for (int k= 1; k<=3 - i; ++k)
                {
                    cout <<" ";
                }
                for (int j = 1; j <= (2 * i - 1); ++j)
                {
                    cout<< num++;
                }
                cout << endl;
            }
        }
        if(c==3)
        {
            int num=1;
            for (i=1;i<=5;i++)
            {
                for(j=1;j<=i;j++)
                {
```

```
                    cout<<num<< " ";
                    num++;
                }
                cout << endl;
            }
        }

        if(c==4)
        {
            int num=1;
            for (i=1;i<=4;i++)
            {
                for (int k=4-i;k>0;k--)
                    cout << "  ";
                for(j=1;j<=i;j++)
                {
                    cout<<num<<" ";
                    num++;
                }
                cout<<endl;
            }
        }
    }
}
```

OUTPUT

```
-------------------------------------------
Enter the type of Pattern you want
-------------------------------------------
 1. Star Pattern
 2. Number Pattern
 3. Continuous Number Pattern
Enter your choice: 1
-------------------------------------------
Enter the shape of Pattern you want
-------------------------------------------
 1. Square
 2. Triangle
 3. Left Triangle
 4. Right Triangle
Enter your choice: 1
-------------------------------------------
* * * * *
* * * * *
* * * * *
* * * * *
* * * * *
```

```
-------------------------------------------
Enter the type of Pattern you want
-------------------------------------------
 1. Star Pattern
 2. Number Pattern
 3. Continuous Number Pattern
Enter your choice: 2
-------------------------------------------
Enter the shape of Pattern you want
-------------------------------------------
 1. Square
 2. Triangle
 3. Left Triangle
 4. Right Triangle
Enter your choice: 1
-------------------------------------------
1 1 1 1 1
2 2 2 2 2
3 3 3 3 3
4 4 4 4 4
5 5 5 5 5
```

```
-------------------------------------------
Enter the type of Pattern you want
-------------------------------------------
 1. Star Pattern
 2. Number Pattern
 3. Continuous Number Pattern
Enter your choice: 3
-------------------------------------------
Enter the shape of Pattern you want
-------------------------------------------
 1. Square
 2. Triangle
 3. Left Triangle
 4. Right Triangle
Enter your choice: 1
-------------------------------------------
1 2 3
4 5 6
7 8 9
```

Observations

- All 12 pattern combinations (3 types × 4 shapes) produced correct visual output.
- The program handles menu navigation correctly and routes to the right pattern block.
- The continuous number counter (num) correctly persists its value across all rows.
- Leading space alignment for right-triangle shapes works as expected.

Limitations

While the program works correctly for its intended purpose, the following limitations exist:

- The pattern size is hardcoded — stars use 5 rows and continuous numbers use 3 rows. Users cannot change the size.

- There is no input validation. Entering a non-integer or out-of-range value causes undefined behaviour.
- The program runs once and exits. There is no option to generate another pattern without restarting.
- Number Pattern Option 2 (Triangle) uses a right-alignment approach that may not perfectly center for all terminal widths.
- No separation between pattern types into functions; all logic is in main(), reducing modularity.
- The program does not support alphabet-based or hollow patterns.

Future Scope

This project can be extended in the following ways in future versions:

- Allow the user to input a custom number of rows so the pattern size is dynamic.
- Add input validation and error handling to prevent crashes on invalid inputs.
- Add a loop to allow the user to generate multiple patterns without restarting.
- Refactor the code by separating each pattern into its own function for better modularity and readability.
- Add new pattern types such as diamond, hollow square, hollow triangle, Pascal's triangle, and alphabet patterns.
- Use color output (platform-specific terminal codes) to make patterns visually more attractive.
- Build a graphical version using a library like SFML or SDL for visual rendering.

Conclusion

The Pattern Generator Program is a successful implementation of a menu-driven C++ console application that demonstrates the practical use of nested loops, conditional statements, sequential counters, and basic console I/O.

Through this project, the core programming concepts of loop nesting, variable scope, and flow control were applied in a visual and intuitive way. The program generates 12 distinct pattern variations across 3 types and 4 shapes, all correctly verified through systematic testing.

This project has strengthened the understanding of how nested loops interact with output positioning to produce two-dimensional patterns on a one-dimensional output stream. The skills and logic patterns applied here form a strong foundation for more complex programming tasks such as matrix operations, game grids, and GUI layout calculations.

Overall, the project meets all its defined objectives and serves as a meaningful academic exercise in introductory C++ programming.